Agent Postmortem Generator

Part 8 - Command Line Interface (CLI)

Design Document

One-line purpose

Provide a thin, deterministic local shell interface for running the postmortem pipeline, validating input traces, and emitting stable exit codes and user-facing status.

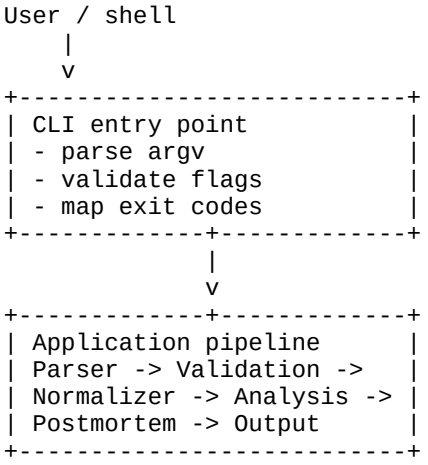
This module is intentionally small: it orchestrates work but never owns parsing, normalization, analysis, or rendering logic.

2. Executive Overview

The Command Line Interface (CLI) is the user entry point for the MVP. It accepts local file paths, command flags, and output preferences; resolves them into a normalized execution request; dispatches the request to the application pipeline; and reports success or failure through deterministic exit codes and concise terminal output.

The CLI exists because the project is meant to be used by engineers on a laptop, from a shell, with no service dependency and no runtime ceremony. It keeps the system accessible while preserving the architecture boundary: the CLI knows about commands, flags, paths, exit codes, and presentation, but it does not know how traces are parsed, normalized, analyzed, or transformed into reports.

Within the system layout, the CLI sits above the output layer and below the human operator. It is the thin adapter between the shell environment and the internal application services.



3. Problem Statement

Users need a reliable way to invoke the tool locally, point it at a trace file, and get a report without memorizing internal package boundaries or importing the project as a library. That sounds trivial until a CLI starts quietly owning business logic, inventing implicit defaults, or emitting unstable output that breaks automation.

This problem is hard because command-line interfaces tend to accumulate accidental complexity. They become a dumping ground for config precedence, ad hoc validation, output formatting, and one-off behaviors that should belong elsewhere. The result is usually a tool that is hard to test, hard to document, and even harder to trust.

The MVP needs this module because the project is explicitly local-first and engineer-facing. Without a clean CLI, the system exists only as a library. That would make initial adoption and day-to-day usage unnecessarily awkward, which is a great way to lose a small open-source project before it starts.

4. Goals

- Expose a small, predictable command surface for running the core workflow and validating trace input.
- Translate shell input into a single internal execution request without leaking framework-specific or presentation-specific details.
- Produce stable exit codes and human-readable messages suitable for both interactive use and scripting.
- Keep the CLI deterministic: the same argv, input file, and environment should yield the same observable behavior.
- Remain thin enough that the CLI can be tested thoroughly with fixture-driven command tests.

5. Non-Goals

Item	Status	Reason

Interactive TUI / REPL	Later version	The MVP is command-driven, not a terminal app with panes, navigation, or live previews.
Remote execution or hosted API	Later version	The project is local-first and has no requirement for a service-backed CLI.
Batch orchestration over many traces	Later version	Multi-run workflows belong to higher-level automation, not the initial shell interface.
Watch mode / file system monitoring	Later version	Continuous monitoring adds state, debounce logic, and a product surface the MVP does not need.
Plugin marketplace or extensibility framework	Later version	The CLI should not become a plugin host before the core workflow is stable.
Framework-specific trace parsing	Never in this module	The CLI only accepts file paths and flags. Framework logic stays in the parser and normalization layers.

6. Responsibilities

Scope	Rule
Own	argv parsing, command routing, flag validation, help/version output, exit codes, terminal status messages, and calling the application pipeline.
Own	basic filesystem checks needed to reach the pipeline, such as path existence and readable input files.
Own	presentation choices at the shell boundary: concise summaries, warnings, error formatting, and optional color handling if implemented.
Never own	trace parsing, schema validation, normalization, analysis, report generation, or any interpretation of trace semantics.
Never own	domain defaults that alter the meaning of the input. The CLI must not decide what a trace event means.
Never own	business logic for incident attribution, root-cause classification, or evidence ranking.

7. Inputs

The CLI accepts a small set of inputs. The MVP should prefer explicitness over convenience.

Input	Required?	Meaning	Validation rule
Positional input path	Required for analyze and validate commands	Local path to a generic JSON trace file.	Must exist, be a regular file, and be readable by the current user.
Command name	Required	analyze, validate, help, version	Must match a supported command exactly.
Output directory	Optional for analyze	Directory where generated artifacts are written.	Must exist or be creatable; write permission is required.
Output format selector	Optional for analyze	Choose markdown, json, mermaid, graph, or a documented subset if the MVP ships narrower.	Must be one of the supported values; defaults must be documented and stable.
Overwrite flag	Optional	Allow replacing existing outputs.	If omitted, existing files must not be silently clobbered.

Assumptions:

- The user is running the CLI locally in a shell, not through a browser or remote session.
- Input traces are already in the project-approved generic JSON format before they reach the CLI.
- The CLI is not responsible for discovering traces automatically. The user supplies the exact path.
- Environment-specific behavior should be minimal and documented. No hidden config file precedence is required for the MVP.

Malformed input handling:

- Unknown flags must produce a usage error with a stable exit code.
- Missing or unreadable input files must fail before any pipeline execution begins.
- Unsupported command names must fail fast and print help text.
- Output path conflicts must be reported clearly rather than overwritten silently.
- Any pipeline error must be surfaced as a command failure with a deterministic message category.

8. Outputs

Output channel	Behavior
Stdout	Short success summaries and help text; no bulk report content unless a future command explicitly routes output to stdout.
Stderr	Usage errors, validation errors, pipeline failures, and warnings.
Exit code	Stable numeric status for shell scripting and automation.
Filesystem artifacts	Generated postmortem outputs written to the selected directory by downstream modules.

The CLI must not generate its own report format. It can print status information about generated artifacts, but the artifact content itself belongs to the output layer.

9. Internal Design

The CLI is best implemented as five internal subcomponents, each with one job.

Subcomponent	Purpose
Entry bootstrap	Initializes the program, configures minimal runtime behavior, and hands off argv to the parser.
Argument parser	Transforms argv into a typed command request and rejects invalid combinations.
Command dispatcher	Routes parsed commands to the correct execution path.
Execution coordinator	Builds the application request, calls the pipeline, and collects the result.
Console renderer	Prints help, errors, warnings, and success summaries with stable wording.

Workflow:

```
argv
  -> parse and normalize flags
  -> validate command and file paths
  -> build execution request
```

```
    -> call application pipeline
    -> map result to terminal output
    -> exit with stable code
```

Important internal decisions:

- Parsing and validation must happen before any domain work is invoked.
- The command surface should be intentionally narrow: a primary analyze command, a validate command, and the standard help/version commands.
- The CLI should avoid hidden config precedence rules in the MVP. Every meaningful setting should be visible in argv.
- All user-facing messages should be deterministic and concise. No timestamps, random identifiers, or non-essential environment dumps by default.
- Any formatting logic must be presentation-only and must not change the meaning of the result.

10. Data Model

Entity	Description
CLIInvocation	Raw process invocation metadata such as argv, process name, working directory, and terminal mode.
CommandRequest	Parsed command name plus validated options such as input path, output directory, and format choice.
ExecutionRequest	A sanitized request passed from the CLI into the application pipeline.
ExecutionResult	Success or failure status returned from the application pipeline.
ExitMapping	Deterministic mapping from result category to process exit code.

This model exists to keep shell concerns and application concerns separate. The CLI needs a request object it can trust, and the downstream pipeline needs an execution request that does not contain raw argv noise.

Relationship view:

```
CLIInvocation
   -> CommandRequest
      -> ExecutionRequest
         -> ExecutionResult
            -> ExitMapping / terminal status
```

11. Algorithms / Processing Logic

The CLI should process input deterministically in the following order.

- Read argv and decide whether the command is one of the supported verbs.
- Parse flags and positional arguments without touching the trace file contents.
- Validate the command-specific shape of the request.
- Check filesystem constraints required to reach the pipeline, such as input readability and output directory writability.
- Construct an execution request and pass it to the application pipeline.
- Render the outcome as a concise success or failure message.
- Translate the final result into a stable exit code.

Edge cases and decision rules:

Case	Required behavior

Unknown command	Print help, do not run the pipeline, exit with usage error.
Duplicate or conflicting flags	Reject immediately with a usage error.
Input file exists but is not readable	Fail before pipeline execution with an input error.
Output path already exists	Fail unless overwrite was explicitly requested.
Pipeline returns partial outputs	Report the partial state clearly and preserve the failure category.
Concurrent invocations on the same output path	No locking semantics are required in MVP; path conflicts should be detected conservatively.

Deterministic behavior requirements:

- Exit codes must not vary based on terminal theme, locale, or TTY status.
- Help text should be stable and ordered so it can be snapshot-tested.
- Warnings should appear in a documented order when multiple validation issues are detected.
- Success summaries should report the same artifact names in the same order every time.
- The CLI must not infer meaning from trace content; only the pipeline may do that.

12. Tradeoffs

Axis	Chosen position
Simplicity vs flexibility	Favor simplicity. Fewer commands are better than a clever but ambiguous interface.
Generality vs specificity	Favor specificity for the MVP. The CLI should work for this project, not every imaginable tool workflow.
Determinism vs heuristic inference	Favor determinism. The CLI should never guess hidden intent from argv patterns.
Interactive convenience vs testability	Favor testability. A thin command runner is easier to verify and harder to break.

The chosen design keeps the CLI boring in the best possible way. A boring CLI is easier to document, easier to automate, easier to support, and much less likely to leak complexity into the rest of the system.

13. Risks and Failure Modes

Risk	Why it matters
Too many command variants	The interface can fragment quickly if every downstream capability gets a new verb too early.
Implicit config precedence	Hidden environment or file-based overrides make behavior hard to explain and test.
Verbose or unstable error text	If messages drift, snapshot tests and user expectations both become brittle.
CLI owning business logic	Any semantic decision made here will eventually duplicate logic in the pipeline.
Path handling edge cases	Relative paths, permission issues, and output collisions are common failure points on laptops.

Hidden complexity to watch:

- Cross-platform filesystem behavior, especially around path normalization and permissions.
- Terminal output wrapping, which can make help and error text look inconsistent if not controlled.
- Exit code semantics, because shell automation depends on them more than humans admit.

14. Testing Strategy

Test layer	Coverage
Unit tests	Argument parsing, validation rules, command routing, exit-code mapping, and message selection.
Integration tests	End-to-end invocation against fixture trace files and expected artifact creation.
Fixture-driven tests	Stable command snapshots for help text, validation errors, and success summaries.
Regression protection	Lock the command surface and exit-code contract so changes are deliberate, not accidental.

Recommended CLI test cases:

- analyze with a valid trace path and output directory
- validate with a valid trace path
- missing input file
- invalid command
- unsupported output format
- existing output path without overwrite
- help and version output
- pipeline failure propagation

15. Implementation Checklist

- Define the supported command surface and document it in help text.
- Implement argv parsing with explicit validation for each command.
- Normalize all paths before dispatch and reject invalid path states early.
- Build an execution request object that contains only sanitized CLI data.
- Map application results to a stable set of exit codes.
- Separate stdout success summaries from stderr diagnostics.
- Add fixture tests for every supported command and every failure class.
- Verify that help text and error messages render consistently on a local terminal.
- Document the exact contract for output directories and overwrite behavior.

16. Future Expansion

Expansion	Why it naturally follows
Batch mode	Run the pipeline against a directory of traces or a manifest file.
JSON machine output	Emit structured command results for automation and scripting.
Shell completions	Generate completion scripts for common shells.
Interactive inspection	Add a TUI or subcommand for browsing reports locally.

Plugin discovery	Expose adapters or command extensions after the core CLI is stable.
STDIN support	Accept piped trace input once trace provenance and error messages can support it cleanly.

These are the right kind of future additions because they extend the shell interface without changing the core contract. Anything that requires new semantic interpretation of traces should still stay below the CLI boundary.

17. Final Recommendation

The design is MVP-ready. The CLI should remain a thin, explicit, deterministic orchestration layer with one primary run command, one validation command, and standard help/version behavior. The main simplification still recommended is to keep the command set small until the core workflow has proven stable in real use.